There is a minimal amount of configuration you can do to a .standard.yml file. The available keys are:

- default_ignores: Defaults to true, if false. Standard will not ignore files that it would normally ignore.
- extend_config: takes the name of additional RuboCop yaml files that can themselves configure RuboCop. The idea is to allow you to run Standard and RuboCop extensions.
- fix: If true, Standard will autocorrect on a regular run, which you can turn off with standard --no-fix.
- format: Sets the formatter for output.
- ignore: Takes a list of file patterns to ignore.
- parallel: If true, Standard will run in parallel.
- ruby_version:: Sets the Ruby version to target; it defaults to the version of Ruby that is running.

You can use Standard as a base for a RuboCop configuration, which you might want to do if you want to use additional RuboCop extensions like the Rails or RSpec cops. In your .standard.yml:

```
extend_config:
        - test_cops.yml
        - rails_cops.yml
```

Those additional files are regular RuboCop files and can do whatever a RuboCop config can do, but the idea of the feature is that you would require and configure a RuboCop extension that Standard does not cover.

Ruby Style in the Large

Ruby style is not just a matter of how you code line-by-line, it also manifests in how you approach problems and how you build solutions. The goal of all of these style recommendations is to allow the code to clearly reflect the intent of the programmer. Taking advantage of what Ruby does best and most simply will make it easier to read and modify your code going forward.

Ruby code is often written with shorter methods and smaller classes than more strongly typed languages. Not only do smaller methods give you more chances to give blocks of code meaningful names, but also having small pieces of functionality made into methods makes it easier to build functionality by combining methods using Ruby syntax such as blocks.

For example, let's say we are sending an email to a user to confirm whether the user is eligible for a promotion and that there are a few different facets to eligibility:

```
def send_promotion_email
  if last_active_date <= 1.year.ago ||
    %w[TX AZ FL].includes?(state) ||
    opted_out?
    return false
  end
  send_the_email
end
```

In this example, the intent of the if condition is somewhat obscured by the syntactic clutter of the three boolean choices. Giving the combined boolean its own method with a meaningful